

New Approximation Algorithms for Longest Common Subsequences

(Extended Abstract)

Lasse Bergroth

Harri Hakonen

Timo Raita¹

Department of Computer Science and
Turku Centre for Computer Science (TUUS)
University of Turku
20520 Turku
Finland
{bergroth,hat,raita}@cs.utu.fi

Abstract

This paper focuses on finding approximations for the longest common subsequence (lcs) of two strings. Most methods which calculate an approximation for the more general problem accepting N ($N \geq 3$) input strings, give typically trivial results for the restricted case under study. Because of the small number of reliable existing heuristics, we introduce several new ones in this survey.

Majority of the presented algorithms give a lower bound for the lcs. Thus they can be used, for example, as a filter to decide quickly, if a more detailed, space- and time-consuming study is needed. A lower bound can also be used to limit the search space of an exact lcs method effectively. The upper bounds complement the information about the true lcs; they form a basis to make a judgement about the reliability of a lower bound. Extensive tests have been carried out to show the strengths of the heuristics and a discussion about their role in various environments is given.

1. Introduction

The application domains of *string comparison methods* range from molecular biology (e.g. the FASTA [15] and BLAST [2] algorithms) to computer science (UNIX diff program), electronics (screen refreshing) and robot movement planning. A conceptually simple definition for the *distance of two sequences* X and Y is to find out the number of

elements in both strings which do not have a correspondent in the other. This can be calculated by determining a longest identical sequence of (not necessarily contiguous) symbols found in both strings.

Let us denote the *input alphabet* by Σ and its size by σ . The input strings X and Y are words of the set Σ^* . Parameter σ can have very different values in various application areas. Typically, if the inputs are natural language text, it is of the order of 100. However, if the lines of text are interpreted as single symbols of the alphabet (as is done in diff), σ is extremely large. On the other hand, when processing biosequences, the size of the alphabet can be as small as 4. A *subsequence* $S[1..t]$ of $X[1..m]$ is obtained by deleting $m - t$ symbols from X . A *common subsequence* (cs) of strings $X[1..m]$ and $Y[1..n]$ – denoted by $cs(X,Y)$ or cs for short – is a subsequence which occurs in both strings. The *longest common subsequence* (lcs) of strings X and Y , $lcs(X,Y)$, is a cs of maximal length. The length of the lcs and the length of its approximation obtained by using a heuristic h are denoted by r and r_h , respectively. The problem can be generalized in a straightforward way to accept N input strings (the N -lcs problem).

Although the 2-lcs problem has gained a lot of attention (see e.g. [8] or [16] for surveys on the topic), heuristic solutions for the 2-lcs problem are rare. In a way this is surprising, since the heuristics have several applications. First, there are situations where a reliable approximation gives good insight to the sequence comparison at hand, avoiding perhaps entirely the execution of the time- and space-consuming exact algorithm. Second, the lower bounds can be used to prune the problem space of exact algorithms significantly. Customizing the exact algorithm to utilize the in-

¹T. Raita acknowledges the support by the Academy of Finland.

formation gathered during the preprocessing phase, the task of determining the lcs can be speeded up. Third, some of the heuristics can be adapted directly to the comparison of biological sequences, in which we are interested not only in the lcs's consisting of exact matches, but also of "evolutionary matches", i.e. pairs of X - and Y -symbols, which are close to each other in evolutionary sense.

A potential source for the lcs-heuristics is the literature on the *shortest common supersequence problem* (scs). In the scs problem we want to find the shortest string in which X and Y occur as subsequences. The 2-scs is closely connected to the 2-lcs problem, because

$$r = m + n - \text{len}(\text{scs}(X, Y)).$$

The $\text{scs}(X, Y)$ can be constructed by determining $\text{lcs}(X, Y)$ and inserting then the remaining symbols from X and Y into the longest common subsequence. This implies that transforming the upper (lower) bounds of the 2-scs heuristics gives lower (upper) bounds for the 2-lcs directly. However, to our knowledge, the general N -scs problem has been studied primarily for the case $N \geq 3$ [10, 11] and the devised techniques give very poor results in the 2-scs case. The theoretical studies of [4, 9] concentrate on finding an expected length for the 2-lcs of two random strings. The assumption of randomness as well as the size of the automata used for deriving the bounds make the approach unacceptable for most practical applications.

The most notable 2-lcs heuristics reported in the literature are those of Chin and Poon [7], Dančik [9], Fraser [10], Jiang and Li [11] and Johtela et al. [12]. Most of the methods have been devised primarily for the general N -lcs problem ($N \geq 3$) and give therefore only a modest approximation result when $N = 2$. A notable exception is the scheme of Fraser [10] which was found independently also by Johtela et al. [12]. It should be noted, however, that in addition to these, the programs calculating an evolutionary distance between two biosequences (e.g. FASTA and BLAST) can also be considered as heuristic 2-lcs algorithms. Methods using the basic ideas of these well-known procedures are described in Section 4.

The structure of the paper is as follows. In Section 2 we describe how to compute three non-trivial upper bounds for the lcs. Two of these methods are based on a relaxation of the original problem. The third is a refinement of a previously published idea. Section 3 describes first three known lower bound algorithms and gives then three new ones. In this connection we show how the original problem can be relaxed in another dimension. The description of each algorithm is quite terse due to space limitations; a detailed discussion is published on some other forum. The experimental framework as well the results and their significance are discussed in Section 4. Finally, Section 5 gives several ways to extend the current work and the basic ideas.

2. Upper bounds

It is well-known that the lcs problem can be formulated as an optimization problem, that of finding the longest path in a graph. The graph is constructed as follows. Each *match*, i.e. a pair of identical symbols $X[i]$ and $Y[j]$, defines a vertex (i, j) in the graph. Auxiliary vertices $(0, 0)$ and $(m + 1, n + 1)$ correspond to the starting vertex and the terminating vertex, respectively. From each vertex (i, j) , we draw a directed edge to vertex (k, l) , iff $i < k$ and $j < l$. In the descriptions of the algorithms it is sometimes advantageous to think that graph is embedded into a two-dimensional tableau, where $X[1..m]$ corresponds to the m rows and $Y[1..n]$ to the n columns of the tableau (w.l.o.g. we will always assume that $m \leq n$). Using this view, we can say e.g. that all edges emanating from (i, j) have their endpoints either in $(m + 1, n + 1)$ or in the rectangle $[i + 1..m, j + 1..n]$.

As an optimization problem, the construction of the lcs has a set of *constraints* which can be *relaxed*. If the relaxation results in an easier problem which retains all solutions to the original one, we get an upper bound for the latter. This general optimization principle is used as the basis for the following upper bound schemes.

U1. Mapping symbols of Σ into equivalence classes

The lcs problem is inherently easier to solve if σ is small. This is due to the fact that when the alphabet size decreases (and the length of the input strings are kept fixed), the average length of the lcs increases [9]. Instead of trying to determine $\text{lcs}(X, Y)$, we change our problem instance by mapping the symbols of Σ into equivalence classes, which represent the elements of the new, reduced alphabet Σ' . Denoting the mapping function by g , we get input strings $X' = g(X)$ and $Y' = g(Y)$ by applying g componentwise to each string. The resulting cs is clearly an upper bound for the actual lcs. The larger the ratio $\sigma'/\sigma \leq 1$ is, i.e. the smaller the equivalence classes are, the better approximation we get. Thus, the ratio can be used to control the tradeoff between the speed and the quality of the result.

To find the approximation, the heuristic must solve an exact problem and therefore its running time is $O(mn)$ in the worst case. However, fixing Σ' (and σ') *a priori* helps us to solve the new problem faster than the original one, because

- (i) the input alphabet is fixed implying that (a) the data structures and (b) the logic used in the exact algorithm can be customized for σ' . The extreme case $\sigma' = 2$ forms a theoretically appealing situation: it is known [1] that the minimum number of decisions under the decision tree model is $O(n)$. Unfortunately, no linear

time algorithm is known for this problem instance.

- (ii) $lcs(X, Y) \leq lcs(X', Y') \leq m$ and by choosing a sufficiently small target alphabet, we create a favourable situation for an exact algorithm which is known to solve fast those problems where r is close to m .

An optimal mapping function distributes the symbols $X[1], \dots, X[m]$ and $Y[1], \dots, Y[n]$ uniformly over all equivalence classes; this minimizes the loss of information caused by the mapping. If the inputs are random, then all equivalence classes have roughly the same number of *different* elements also. Interestingly, the simple mapping

$$a'_i = g(a_i) = (a_i)_2 \bmod 8,$$

i.e. take the three least significant bits from the symbol representation, has a nice property for natural languages (and ASCII code): the most frequent symbols of the skew distribution fall into different equivalence classes. This function is used in the simulation tests below.

U2. Solving independent subproblems

In U2 we use a divide and conquer -approach, where we form subproblems, solve them exactly and then combine the results as if the subproblems were independent of each other. First the symbols of Σ are partitioned into k sets $\Sigma_1, \Sigma_2, \dots, \Sigma_k$. For each of the sets Σ_i , we scan X and Y and pick all the symbols (preserving the order) which belong to the set. As a result, we get strings X'_i and Y'_i ($i = 1, \dots, k$), for which we calculate the length of the exact lcs r_i . The results are collected together simply by forming $\sum_{i=1}^k r_i$. This is the desired upper bound.

There are several ways to partition the original alphabet. We have chosen a scheme which is closely related to the lower bound technique L5: divide the alphabet into two disjoint sets ($k = 2$) so that Σ_1 contains the most frequent symbols and Σ_2 all others. With this selection, we hope that the resulting lcs's interfere less with each other and consequently, the upper bound is better. Moreover, the lcs corresponding to Σ_1 gives a global overview of the actual lcs and is in a sense more important, if we decide to postprocess the lcs's to improve the bound.

The running time of the method is minimized when $len(X'_1) \approx len(X'_2) \approx m/2$ and $len(Y'_1) \approx len(Y'_2) \approx n/2$. For skew distributions, e.g. natural language texts, 6–8 most frequent symbols constitute about half of the occurrences of symbols in both strings (suggesting to solve $lcs(X'_1, Y'_1)$ by an exact algorithm which runs fast for small alphabets). Halving the lengths of the input strings results in approximately half of the work needed for the original problem. The more independent the subproblems are, the better bounds we get. For the typical situation, where the lcs is long, the estimates

are very good.

U3. Loosening the order of the symbols

The trivial upper bound is $\min\{m, n\}$, where m and n are the lengths of the input strings X and Y , respectively. However, a more reasonable bound is obtained with the following scheme. Let us denote the symbols of Σ by $\{a_1, a_2, \dots, a_\sigma\}$ and the frequency of symbol a_i in X by $f_X(a_i)$. A rough estimate of the upper bound is given by

$$r_{U3} = \sum_{i=1}^{\sigma} \min \{f_X(a_i), f_Y(a_i)\}.$$

This is clearly calculated in time $O(m + n + \sigma)$.

Summary on upper bound methods

The methods U1 and U2 use an exact lcs algorithm to find out an upper bound. The approach is reasonable only, if the modified problem has some advantage over the original problem instance. In U1, we can control the expected length of the cs by the size of the target alphabet. A small σ' guarantees that the lcs of the mapped input sequences is quite long suggesting the use of known $O(n(m - r))$ -time algorithms, for example, those of Kumar and Rangan [13] or Wu, Manber, Myers and Miller [18].

Method U2 can be used together with the lower bound technique L5 described below. Experiments have shown that this pair of estimates gives very good insight to the longest common subsequence. As such, they should be elaborated together to improve the execution time.

Method U3 is fast to implement because it scans the input strings only once and it is not sensitive to any parameters used in the generation of the input strings. Note that U3 effectively prunes the occurrences of all those symbols of X which do not occur in Y (and vice versa). In this sense it uses only relevant information to build the estimate. The idea of pruning could be used also to preprocess the input strings and then apply U1 or U2 (or any exact algorithm, for that matter). Other combinations are also possible: we could e.g. map the input strings to a reduced alphabet and then use U3. The hybrid approaches are not studied here, however, because concentrating on the performance of the basic schemes gives us a better understanding how well the underlying mechanisms can find relevant information about the exact lcs.

3. Lower bounds

The description of the lower bound heuristics starts with a review of three previously devised methods, after which we present three new algorithms. Schemes L1 and L2 give simple bounds which are based only on the

frequencies of the symbols. It turns out that using only this information leads to modest performance figures; the position information is of vital importance if we aim at reasonably good estimates.

L1. The symbol of maximum frequency

A crude lower bound [7] is obtained by calculating

$$r_{L1} = \max_{1 \leq i \leq \sigma} \{ \min \{ f_X(a_i), f_Y(a_i) \} \}$$

which can be determined in time $O(m + n + \sigma)$.

L2. Cnt heuristic

In [7], Chin and Poon describe the *cnt* heuristic which first scans the input strings to determine $f_X(a_i)$ and $f_Y(a_i)$ for all symbols $a_i \in \Sigma$. Then a second traversal is prepared by initializing a cursor to the first symbol of each string. If the indicated symbols are equal, the symbol is appended to the cs and both cursors are advanced to the next position. If the symbols are different, the cursor which points to the less frequent symbol is advanced. This is continued until one of the input strings is exhausted. The time complexity of *cnt* is only $O(\sigma + n + m)$. Unfortunately, experiments have shown [12] that resulting lower bound is typically quite weak.

L3. Best next heuristic

In the *best next* heuristic [10] we also scan both input strings in parallel. At each step, we specify the next occurrence (from the position where the cursors point) of each symbol $a_i \in \Sigma$ in both strings. Let us denote these positions by $pos_X(a_i)$ and $pos_Y(a_i)$. The symbol which maximizes $\min\{m - pos_X(a_i), n - pos_Y(a_i)\}$, i.e. the length of the suffix of both strings, is chosen. The cursors are then updated to $pos_X(a_i) + 1$ and $pos_Y(a_i) + 1$, where a_i is the symbol yielding the maximum. This idea was studied independently also in [12], where another approach led to the same basic scheme. The latter paper gave also an implementation, a sound reasoning for its good performance, discussed several alternatives of the heuristic step and made extensive simulations.

The approximation quality of *best next* is very good and it is robust in two important respects: (a) it does not degrade when σ increases, and (b) it is almost insensitive to the distribution of the input symbols. It is also very fast. The asymptotical time complexity of the scheme is $O(m + n + \sigma + \sigma \cdot r_{L3})$.

L4. From an upper bound to a lower bound: the mapping technique

In the mapping method U1 we put symbols into equivalence classes and solved the lcs problem for the mapped

input strings X' and Y' . This lcs, $S'[1..r']$, forms a basis to extract a new lower bound in a straightforward manner: for each $S'[i]$ corresponding to a match $X'[j] = Y'[k]$, check if it is a true match in the original sequences ($X[j] = Y[k]$). If it is, append the symbol to the lower bound sequence. This procedure partitions the common subsequence S' into alternating runs of false and true matches (in the original sequences). The lower bound can be strengthened by taking each run of false matches, prepending and appending it with the corresponding true matches and solving the subproblem. These problems are typically small, so even an exact algorithm can be used. In the experiments section below we shall discuss more about the properties of the subproblems.

The lower bound is obtained in time $O(r')$, if the upper bound sequence is known. Here we must emphasize the difference between the length of the lcs and the actual common subsequence itself: for most exact algorithms the former is solved much faster than the latter. In our case, only the length is needed for the upper bound whereas the calculation of the lower bound requires also the actual sequence $S'[1..r']$. However, we can benefit from this “extra” work when the lower bound is strengthened because we know an interesting fact about each subproblem: *the upper bound for the length of the sub-lcs is given by the number of false matches in that run*. Because of this, we can restrict the search space of the exact method considerably. The variant which strengthens the bound given by L4, is called L4' in what follows.

L5. Reducing the lengths of the input strings

The wide range of applications using lcs algorithms suggests that the nature of the input strings varies significantly in various environments. One of the most important aspects is the distribution of the symbols in input strings. For biosequences it is quite uniform whereas for natural language texts (see e.g. the statistics of the Brown Corpus in [5]) it is very skew. Assuming the latter, we can restrict to the σ' most frequent symbols and still have a good representation of the patterns in the original string. For example, the eight most frequent symbols in the Brown Corpus consist of 62.2% of the length of the Corpus. This observation forms the basis for a new technique to calculate a lower bound.

As a preprocessing stage, we calculate $\min\{f_X(a_i), f_Y(a_i)\}$ for each $a_i \in \Sigma$ and select those σ' symbols for which the value is largest. These symbols form our new alphabet Σ' . The input strings are then shortened by pruning all symbols which belong into $\Sigma \setminus \Sigma'$ and the lcs is determined for the new strings $X'[1..m']$ and $Y'[1..n']$ (note again the close relationship to method U3). Using the above statistics as an example, we have reduced the size of our graph problem by 61%. In addition to being fast, the approach gives us directly a global view of the whole solution. Thus we avoid making heuristic

decisions which are based only on local information (cf. best next heuristic); such decisions are crucial and even a single failure may lead to an unrecoverable solution path. Moreover, the found cs acts like a “backbone” giving fixed points along the original strings and therefore a good starting point if we want to elaborate the result further. There’s no guarantee that points on the backbone belong to the actual lcs, but at least they give a good hint about its direction and important regions. Alphabet Σ' can be determined in time $O(m + n + \sigma \log \sigma')$ by using a heap. Then the lower bound is found in time $O(m + n + m'n')$.

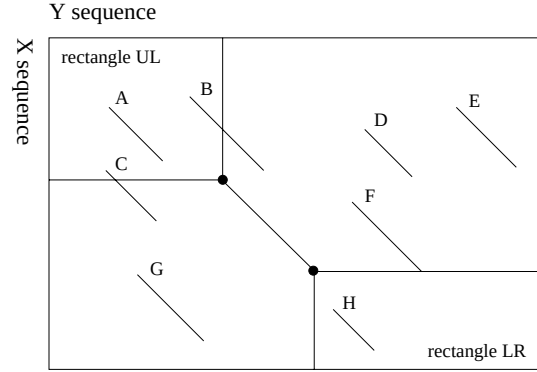
Like in the previous approach, we can try to improve the result between each pair of consecutive backbone points. Again, the earlier steps help in reducing the subproblems further. If (i, j) and (k, l) are consecutive backbone points, which define the region to be studied, all symbols of Σ' can be omitted from the substrings $X[i + 1..k - 1]$ and $Y[j + 1..l - 1]$. Moreover, the nature of the approach implies that the subproblems are very small and therefore simple to solve. This variant is denoted below by L5’.

L6. An approach used in biosequence comparison

The well-known FASTA [15] and BLAST [2] heuristics are designed for efficient database searches. Typically, we give as input a biosequence X and the program scans through all database sequences Y , calculating the edit cost between each (X, Y) -pairs. Finally the database sequences are sorted according to their similarity to X and output on the screen. These programs are essentially also lcs heuristics, because for an appropriate choice of edit costs for symbol insertions, deletions and replacements the problems of edit distance calculations and 2-lcs are identical [8]. However, molecular biology induces several problem-specific factors into the basic edit distance algorithms, such as the cost of gaps and swaps, parametric replacement strategies etc. For this reason, we implemented our own scheme using the ideas of the above programs as a guideline.

The algorithm searches first for all locations (i, j) where X and Y have matching substrings. The preprocessing starts by recording all Y ’s substrings of length q (fixed) along with their positions in Y , scanning then X and maintaining the lengths of identical substrings on each diagonal (of the tableau representation). As a result, we have a list L of triplets (i, j, k) , each representing a k -length match at (i, j) . This list is then scanned, removing all matches which are shorter than $thres$. Parameter $thres$ defines the minimum match length we want to consider. Typically, a safe choice is $thres = 2q$. During the same scan we select the longest match (in case of ties, we select a match which is close to the main diagonal in order to maximize the sizes of the areas which are processed subsequently). The longest match (denoted below by the line segment which

ends in two solid dots) becomes a part of our lower bound and the original problem is divided into two independent subproblems. In the tableau representation the situation is as follows:



List L is scanned, and each time we find a triplet residing in the upper-left (UL) or the lower-right rectangle (LR), we put them to the sublists L_1 and L_2 , respectively. Because of the greedy approach (the chosen line segment is of maximum length), no triplet can span from UL to LR. All matches which lie outside the rectangles (like D, E, F and G) can be discarded. The length and the start/end position of each match which cuts an edge of either rectangle is updated and moved to the corresponding sublist provided that it still satisfies our minimum length constraint. This is continued recursively until all matches are exhausted. As to the time complexity, the initial list of triplets is computed in time $O(\lceil mn / (thres + 1) \rceil)$, because this gives the maximum number of positions to put a run of length $thres$ followed by a single non-match on each diagonal. The common subsequence is built in the worst case by choosing at each recursive step a match which is located in the upper left (or lower right) corner. The size of the remaining rectangle is $(m - thres) \times (n - thres)$, so we have the recursive formula

$$T(m, n) = T(m - thres, n - thres) + O(mn / (thres + 1))$$

for which a rough asymptotic bound is $O(n(\frac{m}{thres})^2)$.

The scheme does not work well, if $thres$ and σ are large and the input strings are random. In this case, the procedure cannot find candidates to start building the common subsequence. The solution is then to lower the value of $thres$ until we get enough candidates into the initial list L . In the experiments below, we use the basic setting $thres = 1$ (and $q = 1$).

Summary on the lower bound methods

The lower bound methods L1, L2 and L3 base their result primarily on the frequencies of the symbols. Due to the small amount of information used to derive the bound,

they execute very fast but the result is strong only in special cases. In fact, it has been shown [7] that any algorithm of this type has a tight lower bound on the performance: the resulting cs may be only of length r/σ . The *best next* method L3 takes advantage also of the symbol position information resulting in stronger bounds.

Methods L4 and L5 are more complex trying to make an overall view of the problem instance and then improve the result locally. In this sense, it is harder for them to get totally lost from the actual common subsequence, no matter how it is spread across the input strings. Method L4 is derived fast from an upper bound which, on the other hand, may take relatively long to determine. However, with this approach we get the upper and lower bounds using the same mechanism, which may be valuable in some situations. Method L5 selects the σ' most frequent symbols and determines the lcs for the pruned input strings. Thus, L1 is a special case of L5 (when $\sigma' = 1$). The complexity analysis of L6 showed that in the worst case it can be very slow. The worst case is met very rarely in practice and by tuning the method for each type of input, we get a reasonably fast result (more of this in the experimental section). Methods L4 and L5 differ from the others in the important respect that using proper parameters we can specify how good approximation we are aiming at; the more we want to use time, the better result we get.

If the lcs is long, L3, L4, L5 and L6 give a good approximation with a high probability. Of these, L5 is somewhat sensitive to the distribution of the input symbols but insensitive to the size of the alphabet. On the other hand, L4 may give weak results if the ratio σ'/σ (σ' is the size of the mapped alphabet) is small. Method L6 relies on finding long stretches of matches in both strings. Because of this, it behaves badly for random (-like) strings. On the other hand, even though the alphabet is large, L6 can find a good approximation, if the length of the lcs is moderately large ($r/m \geq 0.4$).

Similarly to the upper bounds, the lower bound techniques can also be combined or complemented with small add-ons, but we defer the discussion of these variants to the full paper.

4. Simulation tests

Several properties in the input strings have an effect on the performance of the exact and heuristic algorithms. As a general rule, the effect is on the speed for the former and on the strength of the approximation for the latter. The source which generates the input strings can be characterized by

- (a) the distribution of the input symbols,
- (b) the dependencies between symbols, and

- (c) the alphabet size.

The length of the input strings (d) as well as the length of the exact lcs (e) are also important but specific to the applications area. The latter is clearly a function of (a), (b) and (c). Property (b) is closely related to another widely used feature in stringology, that of periodicity. If the minimum length string after which the source begins to repeat the generated sequence is short, the dependencies are very strong and consequently, the lcs is long.

It is evident that the strengths of the upper and lower bounds do not depend heavily on the lengths of the input strings (d). Because the running times can also be normalized by the length, we fixed $n = m = 1000$ (note that some heuristics may confront their worst cases more frequently when the ratio m/n becomes very small; these are not considered here in detail). Furthermore, the two basic input types used (random and natural language text) define directly both the distribution (a) and the dependencies (b). The properties selected were thus (a & b), (c) and (e). Of these, (c) and (e) are not orthogonal; an analysis on random strings shows [9] that the expected length of the lcs depends only on σ (and m). However, the procedure which generates random strings can be controlled so that the length of the lcs can be varied (almost) independently of σ . At this point we would like to remark on the significance of the theoretical result on the string generation procedure. When we built random strings of length 1000 picking one symbol at a time from an alphabet of size 26, the length of the lcs was in the range [311, 328] for 96.6% of all cases. This implies that the generation procedure must be designed very carefully in order to get a sound basis for the experiments (see details in [12]).

Two distribution types were selected quite naturally. A uniform distribution is obtained by the generation of random strings, a skew (Zipfian) distribution is characteristic to natural language texts. We could have used the generator for skew distributions, too. However, we felt that it would be more informative to use natural language texts including in addition higher order dependencies between symbols of test data. This restricts somewhat our test procedure because now the lengths of the lcs's cannot be varied as in the random case. On the other hand, the experimental results below show, that the properties which are inherent to our problem are very stable in natural language texts. The Zipfian input strings were obtained as follows. Two long documents of different languages (English and Finnish) were chosen, both containing technical text. A text was scanned from left to right and parsed into sentences (defined as regions of text which end in a dot). All even-numbered sentences were appended to X and all odd-numbered to Y until their lengths reached m . After one pair of strings have been generated, we continue in the original document from the point where we stopped during the previous iteration. An

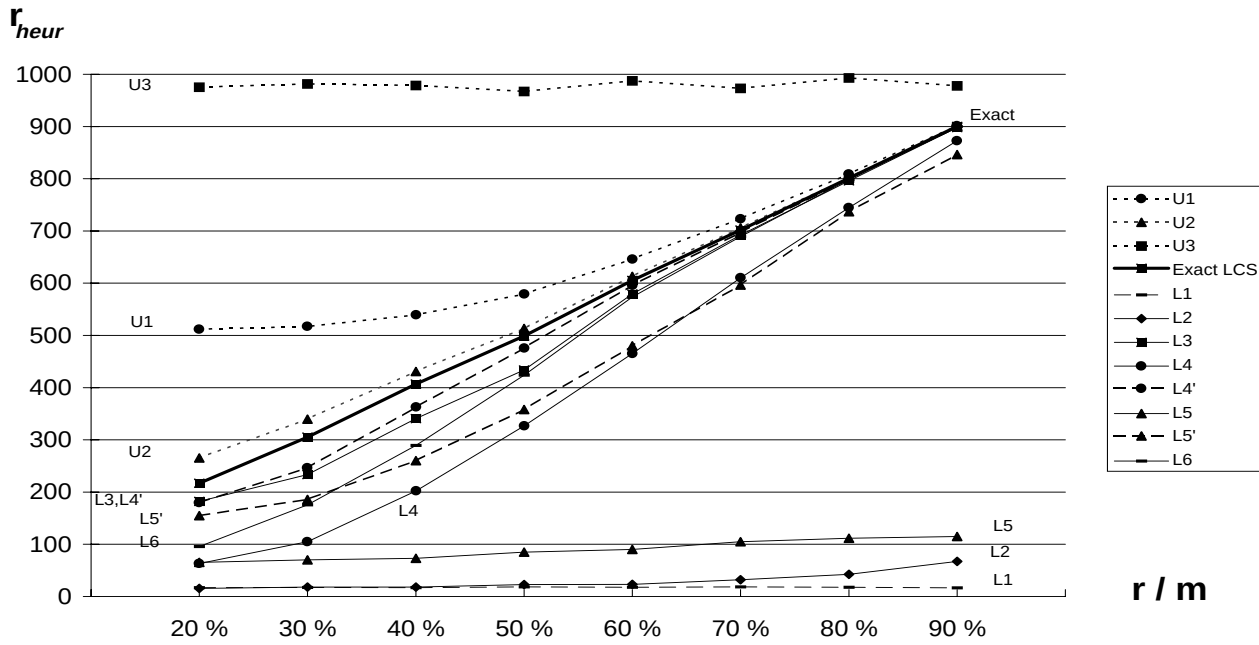


Figure 1. The performance of the upper and lower bound methods as a function of the exact lcs.

interesting statistical fact was the lengths of all lcs's generated this way were in range [367, 460] for English and [382, 517] for Finnish (41 and 47 test pairs, respectively, $m = n = 1000$). In this sense, natural language texts behave like the random ones, although the dependencies between symbols shift the range toward higher values.

4.1. Performance of the heuristics

Figure 1 shows the quality of the approximations as a function of the length of the lcs (ratio r/m) when the input strings are random. The size of the input alphabet is 64. Methods which reduce the alphabet size (L4 and L5) use a target alphabet of size 8 ($\sigma'/\sigma = 1/8$).

As to the upper bounds, we can see that U1 gives reasonable and U2 very good estimates. The lower bound L4 derived from U1, and especially the strengthened variant (L4') give also good approximations. As expected, the result of L3 (best next) is strong. L6 is also very reliable, especially when r is large. It is evident that the use of the most frequent symbols in L5 does not yield good estimates for this type of input, although L5' can improve the result of L5 surprisingly well. The subproblems encountered in L4' and L5' are solved by an exact lcs. Hence, the results of L4' and L5' show the maximal improvement we can achieve continuing from the fixed backbone points determined by the corresponding basic methods. Methods L5 and L5' are apt for skew distributions – this is evidenced clearly by the results of the natural language texts below. The frequency-based

methods U3, L1 and L2 perform poorly in all cases.

When the ratio σ'/σ increases, we lose less information about the original problem, so the methods based on alphabet reduction give better results at the expense of increased execution time. The relative positions of the curves remain almost unchanged.

When experimenting with natural language files, we can in fact select only one point from the above curves determined by the characteristic r value for that language. It was 426 and 438 for English and Finnish, respectively. The results are shown in Table 1.

Table 1. Simulation results for natural language inputs.

Heuristic	English		Finnish	
	r_{heur}	r_{heur}/r	r_{heur}	r_{heur}/r
U1	566	1.33	569	1.30
U2	508	1.19	515	1.18
U3	804	1.89	808	1.84
L1	143	0.34	105	0.24
L2	217	0.51	117	0.27
L3	342	0.80	339	0.77
L4	300	0.70	317	0.72
L4'	369	0.87	383	0.87
L5	341	0.80	346	0.79
L5'	387	0.91	400	0.91
L6	238	0.56	242	0.55

The column r_{heur} gives the average upper/lower bound for

time

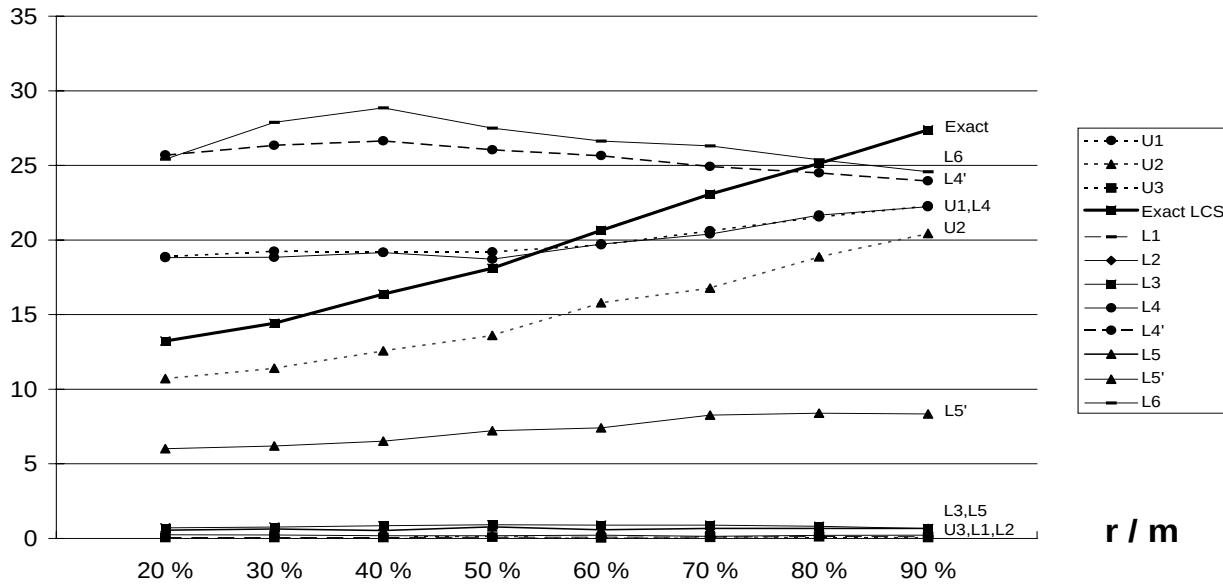


Figure 2. A comparison of the running times of the approximation methods.

all test pairs and the column next to it shows how close to the actual length we get. The variation in the results was very small except for L6, which is sensitive to the value of *thres*. Especially for large values (4-10), the range of the results is quite extensive.

The table shows that the simple methods L1 and L2 don't succeed well. Also, their performance differs in the language used: the results are much better for English than for Finnish. The simple reason is that the distribution of the symbols is significantly skewer in the former. The sophisticated lower bound heuristics have no problem in finding good approximations for strings of both languages; in fact the results are very similar for English and Finnish. The approximations given by L3–L6 can be regarded very good.

4.2. Significance of the results

When we want to compare the execution times of the heuristics with those of an exact algorithm, we are confronted with a dilemma: if the properties of the input strings and the length of the lcs is known *a priori*, we can select the most efficient exact method to solve the problem. The same strategy can be used also when selecting the most appropriate heuristic. Therefore, we have chosen the following procedure to make the comparison fair. The exact algorithm of Chin and Poon [6] forms the basis, i.e. it is the baseline to which we compare the heuristics and it is used always as part of a heuristic, if the method needs an exact algorithm in some subphase. At the current stage of development we

don't have enough information to rank a particular exact method over some other.

Figure 2 depicts the execution times of the methods as a function of r/m . The results are for random strings but the figures for natural language inputs follow the same pattern.

As expected, U1 and its derivatives, L4 and L4' are slow. The execution time of L6 is sensitive to the *thres* value. Threshold value 1, which is used in the tests, results in the slowest mode. Value 2 decreases the time drastically, about one order of magnitude, but gives also inferior approximation. L3 and L5 are quite fast and the result of the latter can be strengthened reasonably quick. The simple methods U3, L1 and L2 are real winners in speed, of course.

Comparison of the performance statistics depicted in Figures 1 and 2 allows us to make some conclusions about the appropriateness of the heuristics in different environments. Obviously, the most important property of heuristic is that it is reliable, that is, it is quite insensitive to changes in the parameters of the generating source. A secondary factor (but almost of the same importance) is the speed. We must keep in mind, though, that when interpreting the effectiveness ratio (approximation quality)/speed our primary goal is reliability. Ordering the methods by decreasing effectiveness ratio, the schemes which clearly stand out are U3, L3, L1, L5 and L2. Of these, reliable are L3, L1 (if r is large) and L5 (if distribution is skew). On the basis of this small analysis, we may conclude that L3 is the preferred method for a wide range of applications. However, it must be remembered that the performance of some methods de-

pend on input parameters which can be tuned for a specific environment.

As to the development of the methods, preliminary tests have shown that if an exact method is prepended with L3 to prune the search space, the running times decrease noticeably. As an example, the augmented Chin & Poon - method performs twice as fast as the original in the case $r = 0.75$, $\sigma = 64$. When r decreases, the gain due to the preprocessing diminishes gradually. The effect of pruning can be seen in those heuristics also which need an exact method. This leads straight to the idea of trying to find the most successful hybrid (either heuristic or exact). However, it is still too early to give any guidelines concerning the most useful building blocks.

5. Discussion and future work

In this paper we have studied several ways to approximate the lcs of two strings. The problem gathers nicely together several interesting and important problems in stringology. Consequently, the presented algorithms have potentially a wide spectrum of application areas. A very important environment to utilize the principles of the schemes is that of searching in a data bank of biosequences. In addition to this, the methods and results can be used directly for the 2-scs problem. Moreover, it seems that many of the presented techniques could be generalized to the N -lcs problem ($N \geq 3$).

The most fruitful direction for future research seems to be in the combination of a heuristic and an exact lcs algorithm. The idea is to find a reliable lower bound in a preprocessing phase so that the problem space of the exact algorithm can be efficiently reduced. The two stages should be “orthogonal” in the sense that the work done during preprocessing can be fully utilized in the exact algorithm. The preliminary results seem promising.

References

- [1] Aho, A.V., Hirschberg, D.S. & Ullman, J.D.: Bounds on the Complexity of the Longest Common Subsequence Problem, *J. of the ACM*, 23(1):1–12, January 1976
- [2] Altschul, S.F., Gish, W., Miller, W. Myers, E.W & Lipman, D.J.: Basic Local Alignment Search Tool, *J. Mol. Biol.*, 215:403–410, 1990
- [3] Argos, P., Vingron, M. & Vogt, G.: Protein Sequence Comparison: Methods and Significance, *Protein Engineering*, 4(4):375–383, 1991
- [4] Baeza-Yates, R.A., Gavalda, R. & Navarro, G.: Bounding the Expected Length of Longest Common Subsequences and Forests, *Third South American Workshop on String Processing, WSP'96*, Recife, Brazil, August 1996, pp. 1–15
- [5] Bell, T.C., Cleary, J.G. & Witten, I.H.: *Text Compression*, Prentice-Hall, 1990
- [6] Chin, F.Y.L. & Poon, C.K.: A Fast Algorithm for Computing Longest Common Subsequences of Small Alphabets Size, *J. of Inf. Proc.*, 13(4):463–469, 1990
- [7] Chin, F. & Poon, C.K.: Performance Analysis of Some Simple Heuristics for Computing Longest Common Subsequences, *Algorithmica*, 12:293–311, 1994
- [8] Crochemore, C. & Rytter, W.: *Text Algorithms*, Oxford University Press, 1994
- [9] Dančík, V.: Expected Length of Longest Common Subsequences, *Ph.D. Thesis*, University of Warwick, September 1994
- [10] Fraser, C.B.: Subsequences and Supersequences of Strings, *Ph.D. Thesis*, University of Glasgow, 1995
- [11] Jiang, T. & Li, M.: On the Approximation of Shortest Common Supersequences and Longest Common Subsequences, *SIAM J. of Comp.*, 24(5):1122–1139, 1995
- [12] Johtela, T., Smed, J., Hakonen, H. & Raita, T.: An Efficient Heuristic for the LCS Problem, *Third South American Workshop on String Processing, WSP'96*, Recife, Brazil, August 1996, pp. 126–140
- [13] Kumar, S.K. & Rangan, C.P.: A Linear-Space Algorithm for the lcs problem, *Acta Informatica*, 24:353–362, 1987
- [14] Myers, E.W.: An Overview of Sequence Comparison Algorithms in Molecular Biology, *Technical Report*, TR 91-29, Dept. of CS, The University of Arizona, Tucson, Arizona
- [15] Pearson, W.R. & Lipman, D.J.: Improved Tools for Biological Sequence Comparison, *Proc. Natl. Acad. Sci. USA*, 85:2444–2448, April 1988
- [16] Stephen, G.A.: String Searching Algorithms, *World Scientific*, 1994
- [17] Ukkonen, E.: Algorithms for Approximate String Matching, *Information and Control*, 64(1-3):100–118, 1985
- [18] Wu, S., Manber, U., Myers, G. & Miller, W.: An $O(NP)$ Sequence Comparison Algorithm, *Inf. Proc. Lett.*, 35:317–323, September 1990